

Al-Aqsa University
Faculty of Computers and IT
Dept. of Info & Computer science



جامعة الأقصى
كمية الحاسبات وتكنولوجيا المعلومات
قسم عمم الحاسوب والمعلومات

Advanced Database Systems

Final Project **Contact Management System**

Presented to: **Eng. Maha AL-Attili**

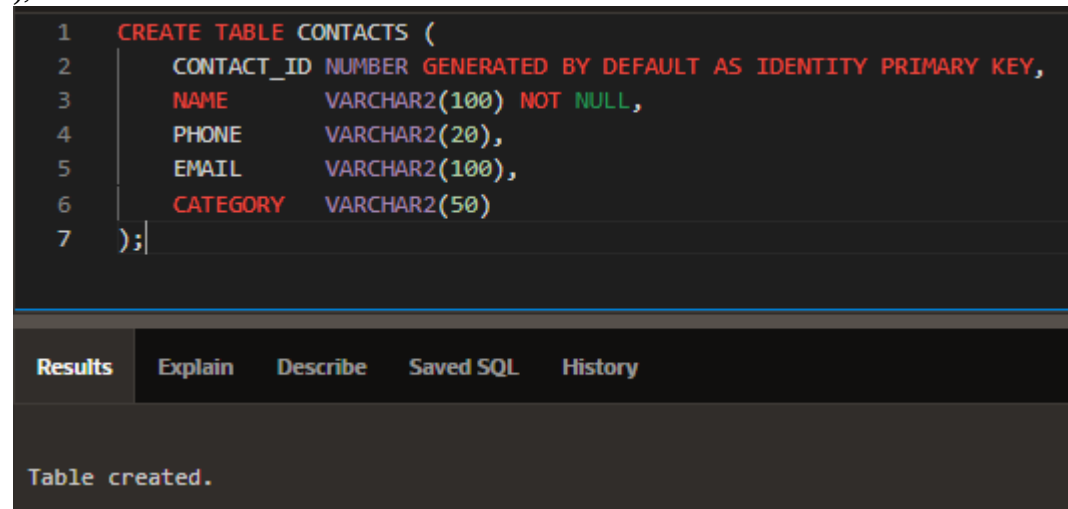
Student name: **Deema Mohammed AL-Maqadma**
Student number: **2320230766**

Semester: First Semester 2025/2026

1. Create CONTACTS Table

We created the CONTACTS table to store contact information. It will serve as the source for saving and managing fields such as Name, Phone, Email, and Category.

```
CREATE TABLE CONTACTS (  
    CONTACT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY  
    PRIMARY KEY,  
    NAME    VARCHAR2(100) NOT NULL,  
    PHONE   VARCHAR2(20),  
    EMAIL   VARCHAR2(100),  
    CATEGORY VARCHAR2(50)  
);
```



The screenshot shows a SQL Developer interface with a dark theme. The top pane contains the SQL code for creating the CONTACTS table. The bottom pane shows the results of the execution, indicating that the table was created successfully.

```
1  CREATE TABLE CONTACTS (  
2      CONTACT_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
3      NAME    VARCHAR2(100) NOT NULL,  
4      PHONE   VARCHAR2(20),  
5      EMAIL   VARCHAR2(100),  
6      CATEGORY VARCHAR2(50)  
7  );
```

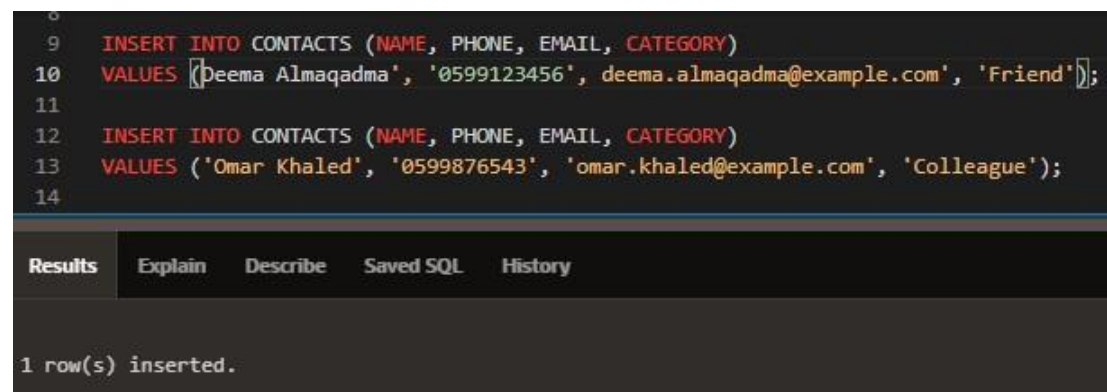
Results Explain Describe Saved SQL History

Table created.

2. Insert Sample Records

We inserted two sample records into the CONTACTS table to test the data storage and ensure the table works correctly.

```
INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)  
VALUES ('Deema AL-Maqadma', '0599123456', deema.almaqadma@example.com',  
'Friend');  
INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)  
VALUES ('Omar Khaled', '0599876543', 'omar.khaled@example.com', 'Colleague');
```



The screenshot shows a SQL Developer interface with a dark theme. The top pane contains the SQL code for inserting two sample records into the CONTACTS table. The bottom pane shows the results of the execution, indicating that 1 row(s) were inserted.

```
9  INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)  
10 VALUES ('Deema Almaqadma', '0599123456', deema.almaqadma@example.com', 'Friend');  
11  
12 INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)  
13 VALUES ('Omar Khaled', '0599876543', 'omar.khaled@example.com', 'Colleague');  
14
```

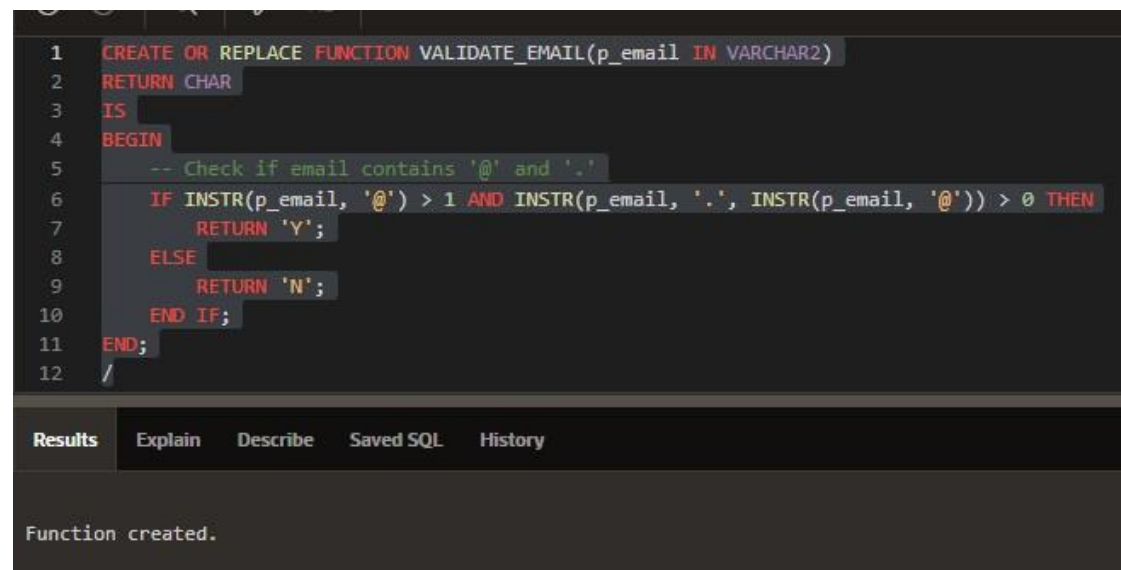
Results Explain Describe Saved SQL History

1 row(s) inserted.

3. Create VALIDATE_EMAIL Function

We created the VALIDATE_EMAIL function to check if an email address has a valid format. It returns 'Y' for valid emails and 'N' for invalid ones.

```
CREATE OR REPLACE FUNCTION VALIDATE_EMAIL(p_email IN
VARCHAR2)
RETURN CHAR
IS
BEGIN
    -- Check if email contains '@' and '.'
    IF INSTR(p_email, '@') > 1 AND INSTR(p_email, '.', INSTR(p_email, '@')) > 0
THEN
        RETURN 'Y';
    ELSE
        RETURN 'N';
    END IF;
END;
/
```



```
1 CREATE OR REPLACE FUNCTION VALIDATE_EMAIL(p_email IN VARCHAR2)
2 RETURN CHAR
3 IS
4 BEGIN
5     -- Check if email contains '@' and '.'
6     IF INSTR(p_email, '@') > 1 AND INSTR(p_email, '.', INSTR(p_email, '@')) > 0 THEN
7         RETURN 'Y';
8     ELSE
9         RETURN 'N';
10    END IF;
11 END;
12 /
```

Results Explain Describe Saved SQL History

Function created.

4. Create ADD_CONTACT Procedure

We created the ADD_CONTACT procedure to insert new contact records into the CONTACTS table using input parameters for Name, Phone, Email, and Category.

```
CREATE OR REPLACE PROCEDURE ADD_CONTACT(
    p_name IN VARCHAR2,
    p_phone IN VARCHAR2,
    p_email IN VARCHAR2,
    p_category IN VARCHAR2
)
IS
```

```
BEGIN
  INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)
  VALUES (p_name, p_phone, p_email, p_category);

  COMMIT;
END;
/
```

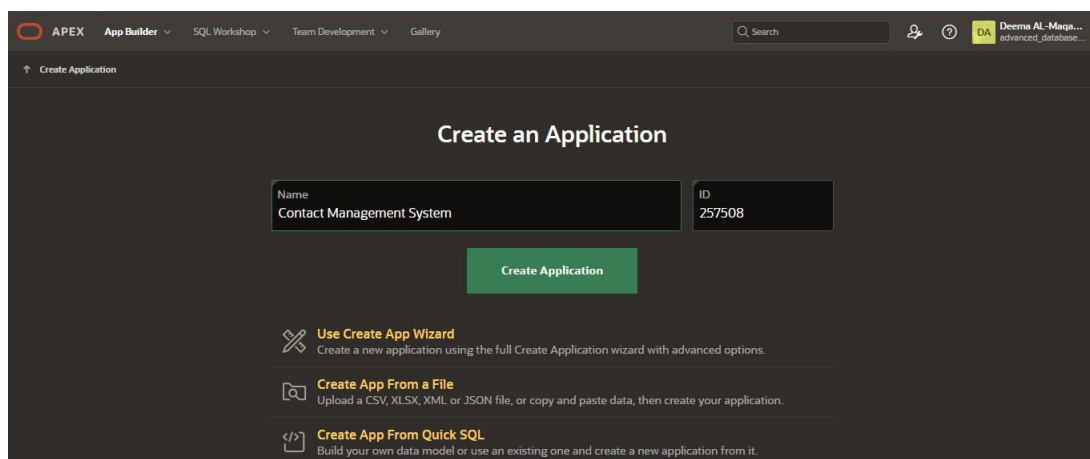
```
14 CREATE OR REPLACE PROCEDURE ADD_CONTACT(
15   p_name   IN VARCHAR2,
16   p_phone  IN VARCHAR2,
17   p_email  IN VARCHAR2,
18   p_category IN VARCHAR2
19 )
20 IS
21 BEGIN
22   INSERT INTO CONTACTS (NAME, PHONE, EMAIL, CATEGORY)
23   VALUES (p_name, p_phone, p_email, p_category);
24
25   COMMIT;
26 END;
27 /
28
```

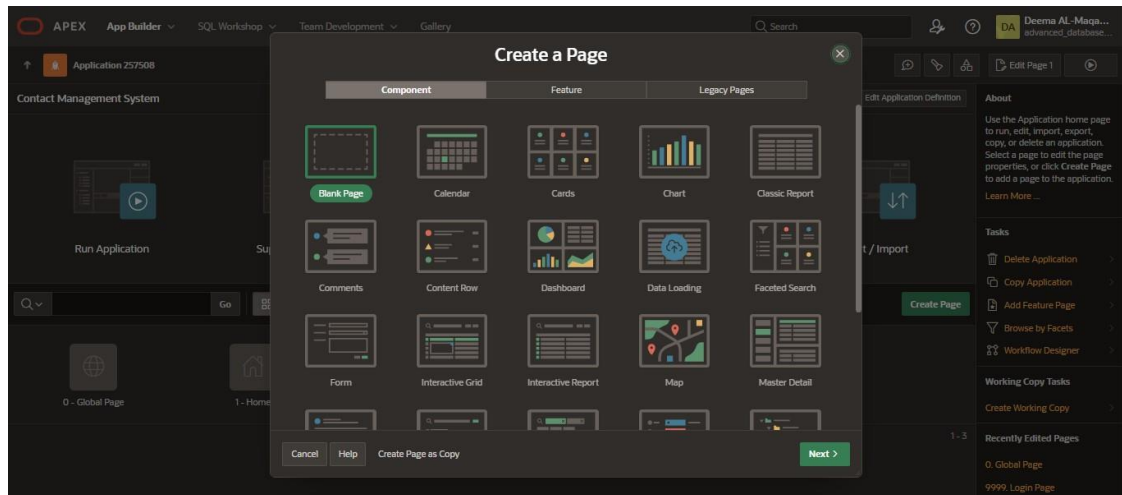
Results Explain Describe Saved SQL History

Procedure created.

5. Create Application in App Builder

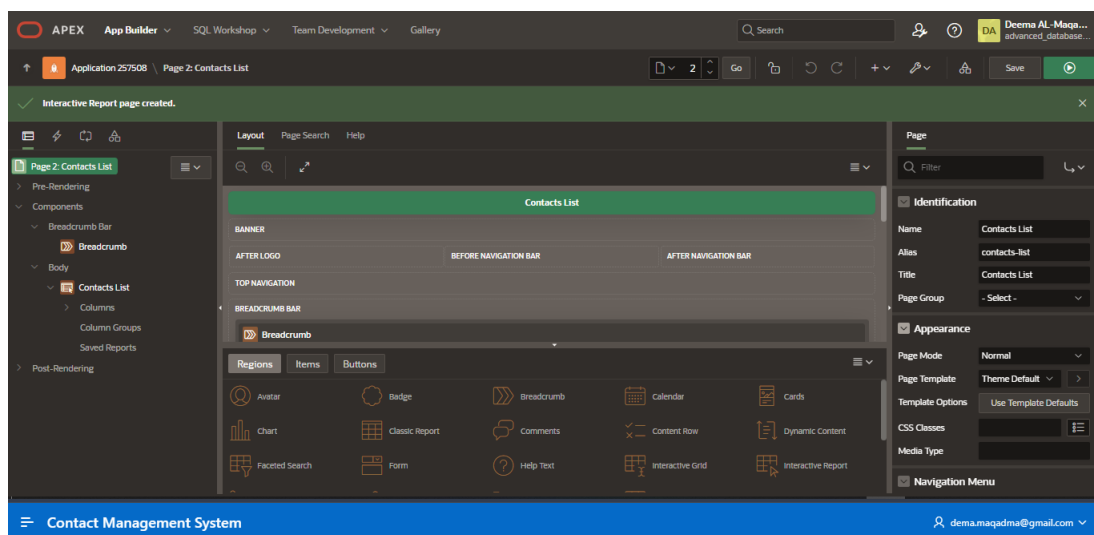
We started building the Contact Management application in Oracle APEX using App Builder. This application will include pages for listing contacts, adding new contacts, and viewing contact details.





6. Create Contacts List Page

We created the first page of the application using an Interactive Report linked to the CONTACTS table. This page displays all contacts, shows the total number of records, and makes each contact name clickable for viewing details.



Contacts List

Q

Go

Actions

Name	Phone	Email	Category
Omar Khaled	0599876543	omar.khaled@example.com	Colleague

1 -

Release 1.0

7. Add Contact Page Buttons and Fields

The Add Contact page automatically displays the fields from the CONTACTS table (Name, Phone, Email, Category). We added two buttons: Add (to save the new contact) and Cancel (to return to the Contacts List page).

The screenshot displays the APEX App Builder interface for configuring the 'Add Contact' page. The top navigation bar shows the application name 'Application 257508' and the page title 'Page 4: Add Contact'. The left sidebar contains a component tree with the following structure:

- Page 4: Add Contact
 - Pre-Rendering
 - Components
 - Breadcrumb Bar
 - Breadcrumb
 - Body
 - Add Contact
 - Region Body
 - P4_CONTACT_ID
 - P4_NAME
 - P4_PHONE
 - P4_EMAIL
 - P4_CATEGORY
 - Close
 - CANCEL
 - DELETE

The central canvas shows the page layout with the following regions:

- BANNER
- AFTER LOGO
- BEFORE NAVIGATION BAR
- AFTER NAVIGATION BAR
- TOP NAVIGATION
- BREADCRUMB BAR
 - Breadcrumb

The right sidebar shows the page properties:

- Page
 - Identification
 - Name: Add Contact
 - Alias: add-contact
 - Title: Add Contact
 - Page Group: - Select -
 - Appearance
 - Page Mode: Normal
 - Page Template: Theme Default
 - Template Options: Use Template Defaults
 - CSS Classes
 - Media Type
 - Navigation Menu

Below the canvas, two detailed views of the 'Add' and 'Cancel' buttons are shown. Each button has a filter, an identification, and a label.

Button	Filter	Identification	Label
Add	Filter	Button Name: Add	Add
Cancel	Filter	Button Name: Cancel	Cancel

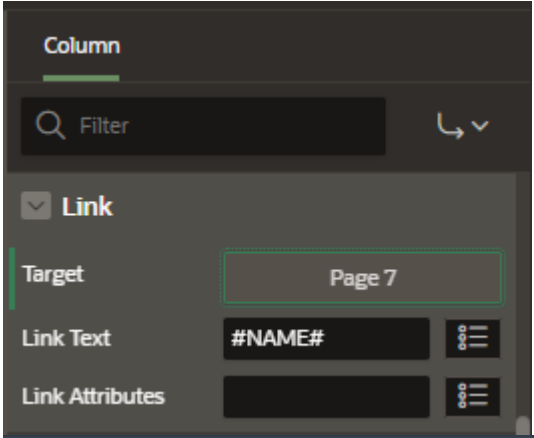
The bottom of the screenshot shows the 'Add Contact' page in a browser window. The page title is 'Add Contact'. The form contains the following fields:

- Name
- Phone
- Email
- Category

At the bottom of the form are two buttons: 'Add' (green) and 'Cancel' (orange).

8. Link Contact Name to Details Page

We configured the Name column in the Contacts List page to open the Contact Details page. The CONTACT_ID value is passed through the column link using the Set Items and With Values options, allowing the details page to display the selected contact's full information.



The screenshot shows the 'Column' configuration interface. Under the 'Link' section, the 'Target' is set to 'Page 7', the 'Link Text' is '#NAME#', and 'Link Attributes' are empty.

Contacts List

Name	Phone	Email	Category
Omar Khaled	0599876543	omar.khaled@example.com	Colleague

1 - 1

Contact Details

Contact Details

Name
Omar Khaled

Phone
0599876543

Email
omar.khaled@example.com

Category
Colleague

Delete Apply Changes

9. RT 4.1 – Validations

We added two validations to the Add Contact page. First, we marked the Name field as required to ensure users cannot submit an empty name.

The screenshot shows the 'Add Contact' page in a contact management system. The 'Name' field is highlighted with a red border and an error message: 'Name must have some value.' The 'Phone' field contains '0568764837', the 'Email' field contains 'omar.ma@gmail.com', and the 'Category' field contains 'Engineer'. A yellow error notification box at the top right states '1 error has occurred' and 'Name must have some value.' The 'Add' button is green and the 'Cancel' button is orange.

Page Item

Filter

Validation

Value Required ☒

Maximum Length 100 characters

Contact Management System

Add Contact

Add Contact

Name

Phone
0568764837

Email
omar.ma@gmail.com

Category
Engineer

Add

Cancel

1 error has occurred
• Name must have some value.

App 257508 Page 4 Session Debug Quick Edit Customize

10. RT 4.1 – Validations

Second, we created a PL/SQL validation that calls the `VALIDATE_EMAIL` function to verify the email format. If the function returns 'N', an error message is displayed to the user.

The screenshot displays the Oracle APEX interface for configuring a validation. The 'Validation' panel is open, showing the following details:

- Name:** `VALIDATE_EMAIL`
- Execution:** (Expanded)
- Validation:** (Expanded)
 - Editable Region:** - Select -
 - Type:** Expression
 - Language:** PL/SQL
 - PL/SQL Expression:** `RETURN VALIDATEEMAIL(:P2EMAIL) = 'Y';`
 - Always Execute:** (Toggle switch is off)
- Error:** (Expanded)
 - Error Message:** `Invalid email format.`

Below the configuration panel, the 'Add Contact' form is visible. The form contains the following data:

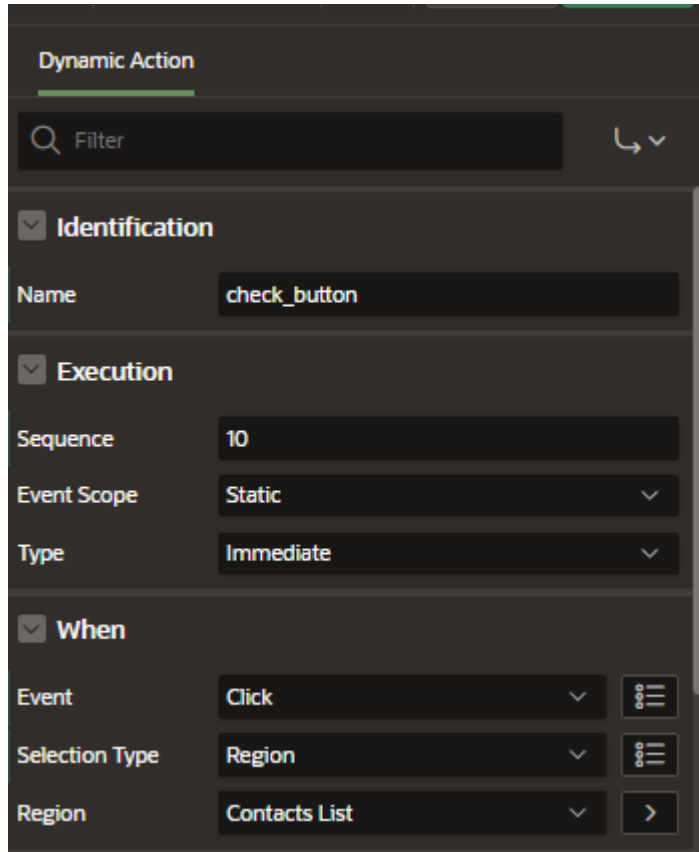
- Name:** Deema Mohammed AL-Maqadma
- Phone:** 0599879617
- Email:** deema.maqadma.com
- Category:** Computer Science

The 'Add' button is highlighted in green, and the 'Cancel' button is highlighted in orange. A yellow error message banner at the top right of the form states: '1 error has occurred' and 'Error processing validation.'

The bottom status bar shows the application name 'Contact Management System', the page number 'Page 4', and various utility icons.

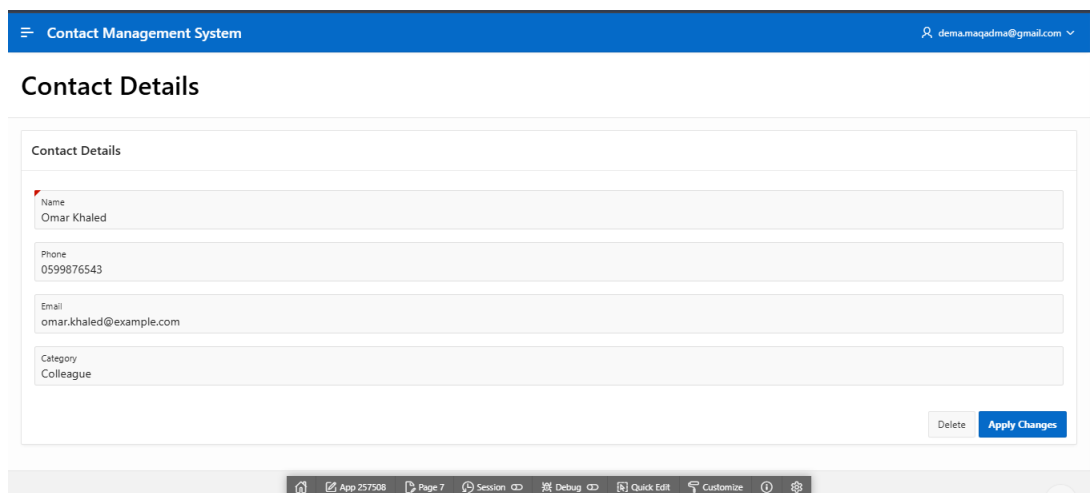
11. RT 4.2 – Dynamic Action

We created a Dynamic Action on the Contacts List page. When the user clicks on a contact name, the system navigates to the Contact Details page and passes the CONTACT_ID, allowing the full details of the selected contact to be displayed.



The screenshot shows the 'Dynamic Action' configuration window. It has a 'Filter' search bar at the top. Below it are three sections: 'Identification', 'Execution', and 'When'. In the 'Identification' section, the 'Name' is set to 'check_button'. In the 'Execution' section, 'Sequence' is 10, 'Event Scope' is 'Static', and 'Type' is 'Immediate'. In the 'When' section, 'Event' is 'Click', 'Selection Type' is 'Region', and 'Region' is 'Contacts List'. Each setting in the 'When' section has a menu icon to its right.

Section	Property	Value
Identification	Name	check_button
Execution	Sequence	10
	Event Scope	Static
	Type	Immediate
When	Event	Click
	Selection Type	Region
	Region	Contacts List



The screenshot shows the 'Contact Details' page of a 'Contact Management System'. The page has a blue header with the system name and a user profile. The main content area is titled 'Contact Details' and contains a form with the following fields: Name (Omar Khaled), Phone (0599876543), Email (omar.khaled@example.com), and Category (Colleague). At the bottom right of the form are 'Delete' and 'Apply Changes' buttons. The footer of the application shows various icons and labels like 'App 257508', 'Page 7', 'Session', 'Debug', 'Quick Edit', and 'Customize'.

Field	Value
Name	Omar Khaled
Phone	0599876543
Email	omar.khaled@example.com
Category	Colleague

12. RT 4.3 – Authorization

We created an Authorization Scheme in Shared Components using a PL/SQL function that allows only the ADMIN user to access the Add Contact page. The scheme was then applied to the page's security settings to restrict access for other users.

The screenshot shows the Oracle APEX interface. The top part displays the 'Create Authorization Scheme' dialog box. The dialog has a 'Details' tab selected. It shows the following configuration:

- Application: 257508 Contact Management System
- Name: Admin Only Access
- Scheme Type: PL/SQL Function Returning Boolean
- PL/SQL Function Body: `RETURN (UPPER(:APP_USER) = 'ADMIN');`

Below the dialog, the 'Authorization Schemes' list is visible. It contains two entries:

Name	Type	Caching	Subscribed From	Subscription Status	Subscribers	Updated	Copy
Admin Only Access	PL/SQL Function Returning Boolean	Once per session				2 seconds ago	
Administration Rights	PL/SQL Function Returning Boolean	Once per page view				2 days ago	

The bottom part of the screenshot shows the 'About' section on the right, which provides information about Authorization Schemes, including their purpose and common types (Exists, Not Exists SQL Queries, and PL/SQL Function Returning Boolean).

Page

Filter

Security

Authorization Scheme

Admin Only Access

Authentication

Page Requires Authentication

Deep Linking

Application Default

Page Access Protection

Arguments Must Have Checksum

Form Auto Complete

Off

Browser Cache

Application Default



Access Denied

Access denied by Page security check

Technical Info (only available for developers)

1. is_internal_error: true
2. apex_error_code: ACCESS_DENIED_SIMPLE
3. component.type: APEX_APPLICATION_AUTHORIZATION
4. component.id: 137731880639174345499
5. component.name: Admin Only Access
6. error_backtrace:

```
----- PL/SQL Call Stack -----
  object      line      object
  handle      number    name
0x561c5c2be0  1058     package body APEX_240200.MW_FLOW_ERROR.INTERNAL_GET_ERROR
0x561c5c2be0  1127     package body APEX_240200.MW_FLOW_ERROR.INTERNAL_ADD_ERROR
0x561c5c2be0  1611     package body APEX_240200.MW_FLOW_ERROR.RAISE_INTERNAL_ERROR
0x55f9b4ca70  978      package body APEX_240200.MW_FLOW_AUTHORIZATION.IS_AUTHORIZED
0x55fa08aa48  3133     package body APEX_240200.MW_FLOW.SHOW
0x55fa08aa48  5542     package body APEX_240200.MW_FLOW.RESOLVE_FRIENDLY_URL
0x55fa08aa48  5606     package body APEX_240200.MW_FLOW.ROUTE_FRIENDLY_URL
0x55dc0eae98      6      anonymous block
```

OK

Conclusion :

In this project, we developed a Contact Management System using Oracle APEX. The work included creating the database objects (tables, functions, and procedures), building the application with three main pages (Contacts List, Add Contact, and Contact Details), and implementing validations, dynamic actions, and authorization schemes. The application allows users to manage contacts efficiently, ensures data integrity through validations, provides interactive navigation between pages, and restricts access to sensitive functionality using security controls. This project demonstrates the integration of SQL development with APEX application building to deliver a complete, functional solution.

"واخر دعواهم ان الحمد لله رب العالميه"